PROYECTO FINAL

SISTEMA DE GESTIÓN DE VENTAS Y FINANCIAMIENTO PARA CONCESIONARIA

OBJETIVO

El sistema resuelve la gestión de ventas de vehículos y planes de financiamiento de una concesionaria. Permite:

- Administrar clientes, vehículos, marcas y planes de pago.
- Registrar ventas, calcular anticipos y montos financiados.
- Calcular cuotas y registrar pagos realizados.
- Consultar el estado de cada operación.
- Controlar el stock disponible.
- Identificar cuotas vencidas.
- Registrar saldos pendientes.

Además, proporcionará estadísticas de ventas y recaudación para facilitar el seguimiento de la actividad comercial y financiera de la empresa.

Alcances del sistema

Gestión de marcas

- CRUD
  - Registrar, modificar y consultar marcas y el listado de vehículos de una marca.
- Lógica de negocio
  - Eliminar una marca, solo si no tiene vehículos asociados.

Gestión de clientes

- CRUD
  - Registrar, editar y consultar clientes.
- Lógica de negocio
  - Validar que el DNI y el correo electrónico no estén duplicados.
  - Eliminar un cliente, solo si no tiene ventas asociadas.

Gestión de vehículos

- CRUD
  - Registrar, modificar y consultar vehículos (marca, tipo de vehículo, tipo de motor, modelo, año, precio y stock).
  - Consultar el listado de vehículos disponibles de una marca y la información de un vehículo determinado.
- Lógica de negocio
  - Un vehículo está disponible para la venta cuando su stock es mayor a cero.

- Eliminar un vehículo, solo si no tiene ventas asociadas.

Gestión de tipos de vehículo

- CRUD
  - Registrar, modificar y consultar tipos de vehículo (por ejemplo: auto, camioneta, utilitario) y el listado de vehículos de un tipo.
- Lógica de negocio
  - Eliminar un tipo de vehículo, solo si no tiene vehículos asociados.

Gestión de tipos de motor

- CRUD
  - Registrar, modificar y consultar tipos de motor (por ejemplo: nafta, diésel, GNC, híbrido, eléctrico) y el listado de vehículos de un tipo de motor.
- Lógica de negocio
  - Eliminar un tipo de motor, solo si no tiene vehículos asociados

Gestión de planes de financiamiento

- CRUD
  - Registrar planes (nombre, cantidad de cuotas, porcentaje de interés total y porcentaje de anticipo).
  - Consultar, modificar y eliminar planes.
- Lógica de negocio
  - Eliminar un plan, solo si no tiene ventas asociadas.
  - El plan no contiene datos del cliente ni del vehículo: esos datos se cargan en la venta.

Gestión de ventas

- CRUD
  - Registrar nuevas ventas, seleccionando cliente, vehículo y plan de financiamiento. La fecha de operación se registra automáticamente al momento de crear la venta.
  - Consultar ventas.
  - Una venta registrada no se modifica ni se elimina.
- Lógica de negocio
  - Cálculo de anticipo, monto financiado, interés e importe de las cuotas.
  - Generación automática de las cuotas.
  - Descuento de una unidad del stock del vehículo.
  - El anticipo se registra como parte de la operación de venta (no como un pago de cuota).
  - Al registrar una venta, el sistema guardará un resumen de los valores al momento de la operación: el precio del vehículo, el monto del anticipo, el monto financiado y el monto total con intereses. Esto asegura que las cuotas no se modifiquen si luego cambia el precio del vehículo o las condiciones del plan.
- Consulta consolidada / reporte

- Registro completo de una venta: detalle, fecha, plan, cuotas, importes abonados y saldo pendiente.

Gestión de cuotas y pagos

- CRUD
  - Registrar pagos (totales o parciales) correspondientes a una cuota, con su fecha e importe.
  - Consultar las cuotas de una venta y los pagos asociados.
- Lógica de negocios
  - Calcular el saldo pendiente de cada cuota (importe menos pagos recibidos).
  - Identificar cuotas pagadas, pendientes y vencidas.
  - Listar las cuotas vencidas con saldo pendiente.
  - Impedir que un pago supere el saldo pendiente

Eliminación de registros relacionados: los registros que tienen ventas o vehículos asociados, según corresponda, no pueden eliminarse. El sistema informa el motivo y no realiza la operación.

Salidas del sistema

El sistema permitirá obtener listados e informes gráficos para consultar y analizar la información relacionada con las ventas, los vehículos, los clientes y el financiamiento de la concesionaria.

- Listados para consultas

1. *Listado de clientes*: permitirá consultar los clientes registrados y buscar por DNI o apellido.

2. *Listado de vehículos:*
   a) *General*: permitirá consultar los vehículos registrados, con su marca, tipo de vehículo, tipo de motor, modelo, año, precio y stock disponible.
   b) *Vehículos disponibles*: mostrará los vehículos que tengan stock mayor a cero, con posibilidad de filtrar por marca.
   c) *Vehículos sin stock*: mostrará los vehículos que no tengan unidades disponibles para la venta.

3. *Listado de planes de financiamiento*: permitirá consultar los planes registrados, con su cantidad de cuotas, porcentaje de anticipo y porcentaje de interés.

4. *Listado de ventas*: permitirá consultar las ventas realizadas, con su fecha, cliente, vehículo y plan de financiamiento. Se podrá consultar por período.
   a) *Detalle de una venta*: mostrará los datos de la operación, el precio del vehículo al momento de la venta, el anticipo, el monto financiado, el total con intereses, las cuotas generadas, los pagos recibidos y el saldo pendiente.

5. *Listado de cuotas*

a) *De una venta*: mostrará el número de cada cuota, su importe, fecha de vencimiento, pagos realizados, saldo pendiente y estado (pagada, pendiente o vencida).
b) *Vencidas*: mostrará las cuotas cuya fecha de vencimiento haya pasado y que todavía tengan saldo pendiente, incluyendo los datos del cliente y de la venta correspondiente.
c) *Por vencer*: permitirá consultar las cuotas con saldo pendiente cuyos vencimientos se encuentren dentro de los próximos siete o treinta días.

6. *Listado de pagos recibidos*: permitirá consultar los pagos registrados, indicando su fecha, importe, cuota y venta correspondiente, con posibilidad de filtrar por período.

7. *Listado de deudas pendientes por cliente*: mostrará los clientes que tengan saldos pendientes y el importe total adeudado por cada uno.

Informes gráficos

1. *Ventas por mes*:
   a) *Cantidad:* mostrará la evolución mensual de las ventas realizadas.
   b) *Importe total de las ventas por mes*: mostrará la evolución mensual de las ventas, sumando el precio del vehículo registrado en cada venta.
2. *Recaudación mensual*: mostrará el importe total recibido por anticipos y pagos de cuotas, agrupado por mes. Sumará los anticipos registrados en las ventas del mes y los pagos de cuotas recibidos en ese mismo mes.
3. *Ventas por marca*: mostrará la cantidad de vehículos vendidos de cada marca.
4. *Ventas por tipo de vehículo*: mostrará la cantidad de vehículos vendidos de cada tipo de vehículo.
5. *Ventas por tipo de motor*: mostrará la cantidad de vehículos vendidos de cada tipo de motor.
6. *Ventas por plan de financiamiento:* mostrará la cantidad de ventas realizadas con cada plan.
7. *Estado de las cuotas*: representará la cantidad de cuotas pagadas, pendientes y vencidas.

Los resultados de los informes gráficos se obtendrán mediante consultas SQL agrupadas y se representarán gráficamente para facilitar su interpretación.

Reglas de negocio

1. Cálculo del financiamiento

El sistema utilizará las siguientes fórmulas:

- Anticipo = precio del vehículo × porcentaje de anticipo / 100.
- Monto financiado = precio del vehículo − anticipo.

- Interés total = monto financiado × porcentaje de interés / 100.
- Total financiado con interés = monto financiado + interés total.
- Importe de cada cuota = total financiado con interés / cantidad de cuotas.

Se utilizará un interés total fijo sobre el monto financiado. El sistema no calculará intereses por mora ni aplicará sistemas de amortización bancarios.

2. Generación de cuotas

Las cuotas se generarán automáticamente al registrar la venta. Tendrán importes iguales y vencimientos mensuales consecutivos, tomando como referencia la fecha de la operación.

3. Estado de las cuotas

El estado de una cuota se determinará según su saldo y su fecha de vencimiento:

- Pagada: no tiene saldo pendiente.
- Pendiente: tiene saldo pendiente y todavía no venció.
- Vencida: tiene saldo pendiente y su fecha de vencimiento ya pasó.

Estos estados se calcularán a partir de los datos de la cuota y sus pagos, sin necesidad de almacenarlos como estados permanentes.

> ***Aclaración sobre datos derivados:***
>
> *En el diseño de la base de datos se aplica el principio de no almacenar información que pueda ser calculada a partir de otros datos (datos derivados), para evitar inconsistencias. Almacenar una columna con el estado requeriría un proceso automático que actualice las filas diariamente. Para evitar esta redundancia, el sistema no guarda el estado como un dato permanente, sino que lo calcula dinámicamente en el momento de la consulta, comparando el saldo pendiente con la fecha de vencimiento y la fecha actual.*

4. Integridad de las operaciones

El sistema deberá validar la disponibilidad de stock, los importes de las operaciones y los pagos registrados. Las operaciones que involucren cambios relacionados deberán mantener la integridad de los datos ante posibles errores.

5. Entidades principales previstas

El sistema contará con nueve entidades: Marca, Tipo de vehículo, Tipo de motor, Cliente, Vehículo, Plan de financiamiento, Venta, Cuota y Pago.

- Venta: Almacena los datos de la operación y el resumen de los valores calculados al momento de la venta: precio_vehiculo, monto_anticipo, monto_financiado y monto_total_con_interes.
- Cuota: Almacena el número de cuota, el importe y la fecha de vencimiento. No almacena un estado (como "pagada" o "vencida"), ya que este se calcula dinámicamente a partir de los pagos recibidos y la fecha actual.

- Pago: Almacena los pagos realizados por el cliente para saldar las cuotas.

Las relaciones, los atributos y las restricciones de cada entidad se definirán en el diagrama entidad-relación y el modelo relacional.

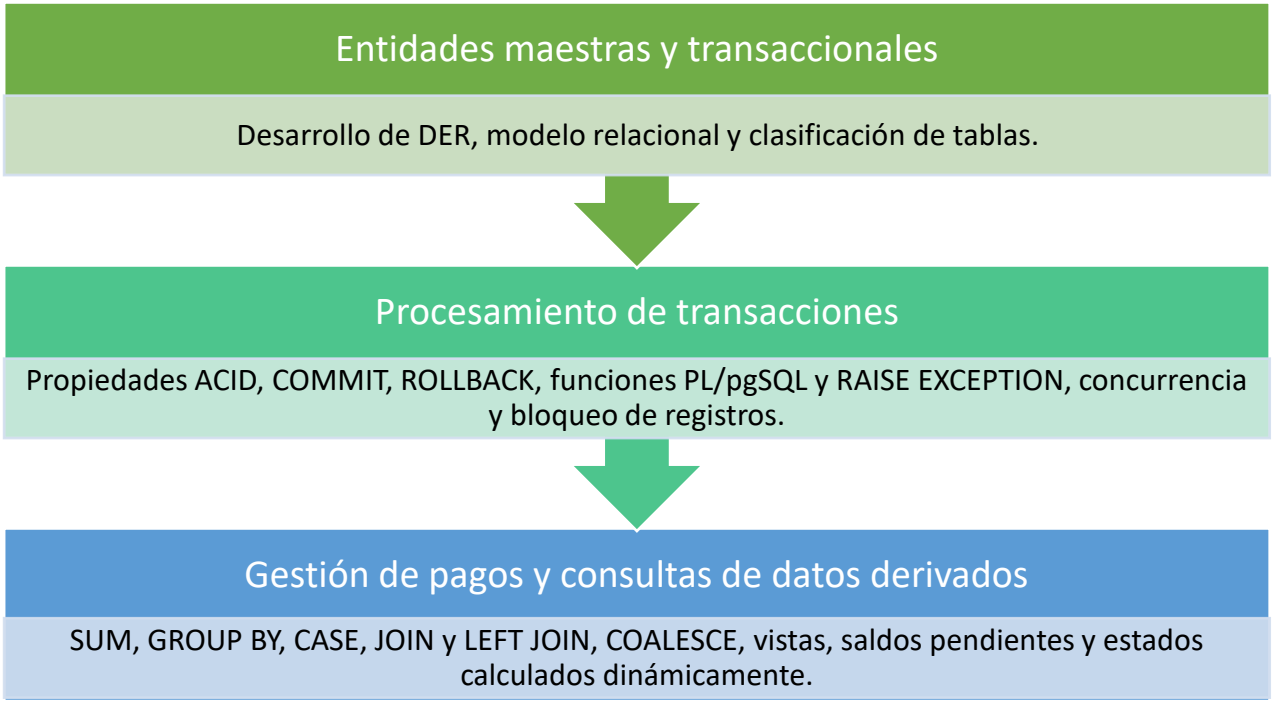
Tecnologías previstas

- Backend: Node.js y Express.
- Base de datos: PostgreSQL.
- Frontend: HTML, CSS y JavaScript.
- Estadísticas: Chart.js.
- Pruebas de endpoints: Postman.

No incluye: usuarios y contraseñas, intereses por mora, cancelación anticipada, evaluación crediticia ni facturación.

BASE DE DATOS

Para ordenar el trabajo en la base de datos, se usó el siguiente esquema:

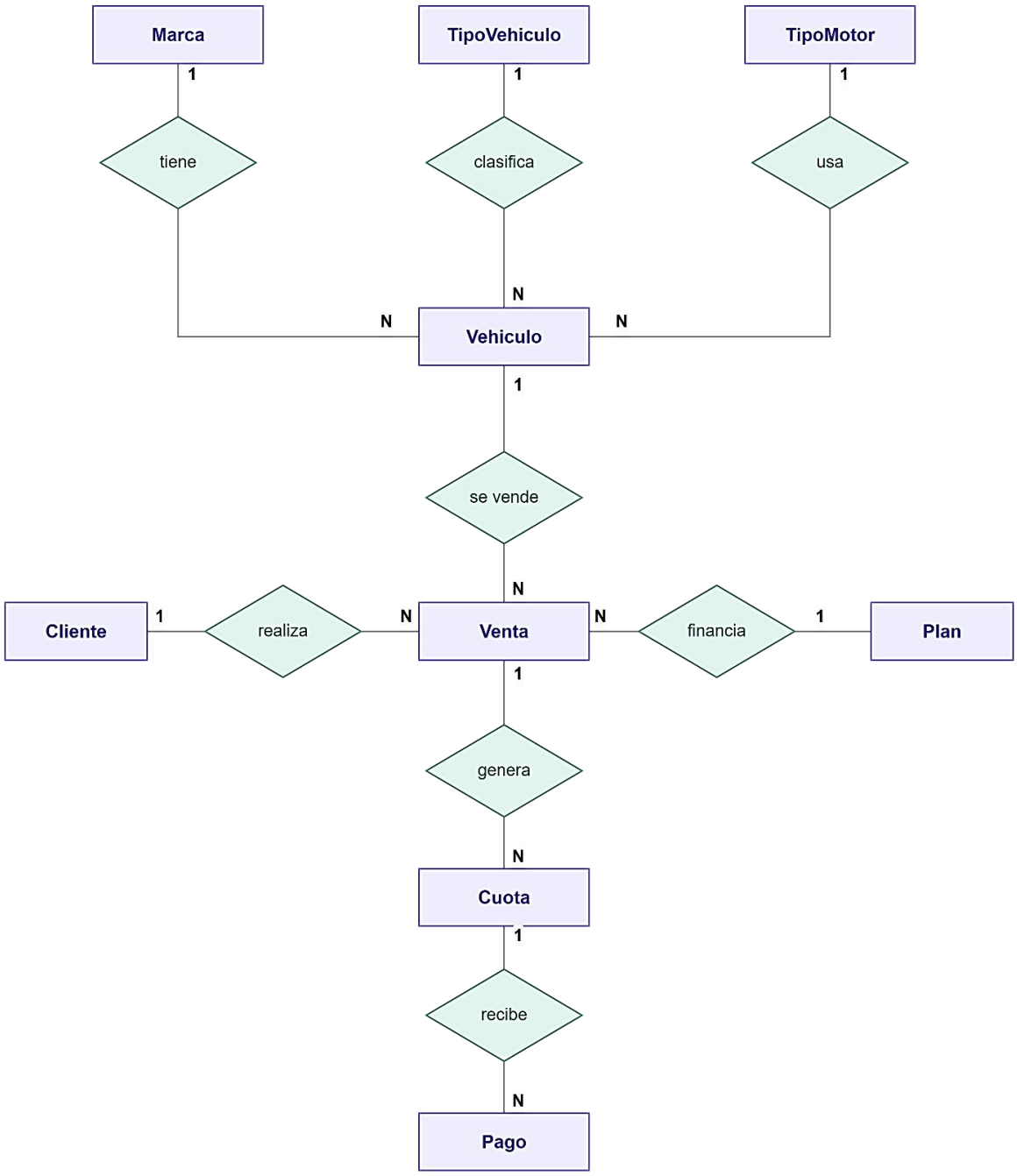


DER Y MODELO RELACIONAL

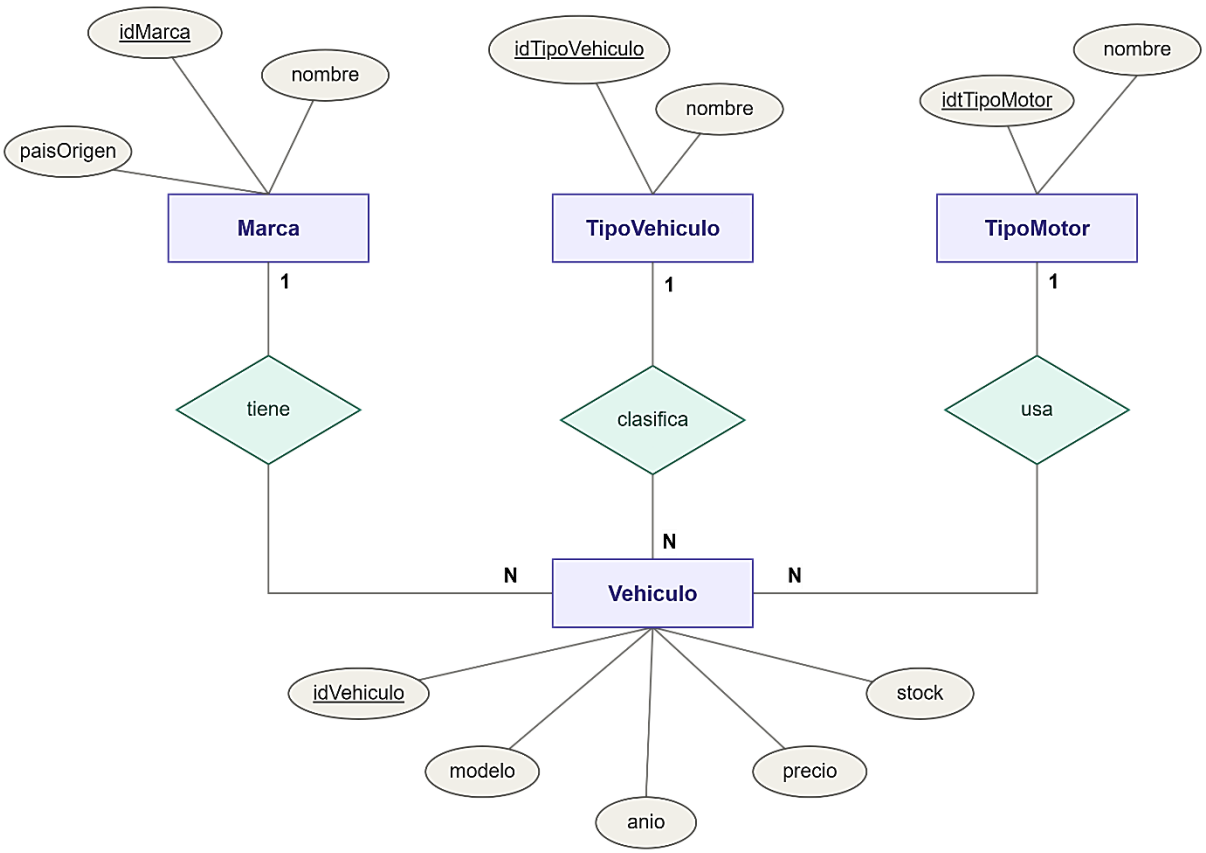
1. DIAGRAMA ENTIDAD – RELACIÓN | DER

DIAGRAMA ENTIDAD - RELACION DER

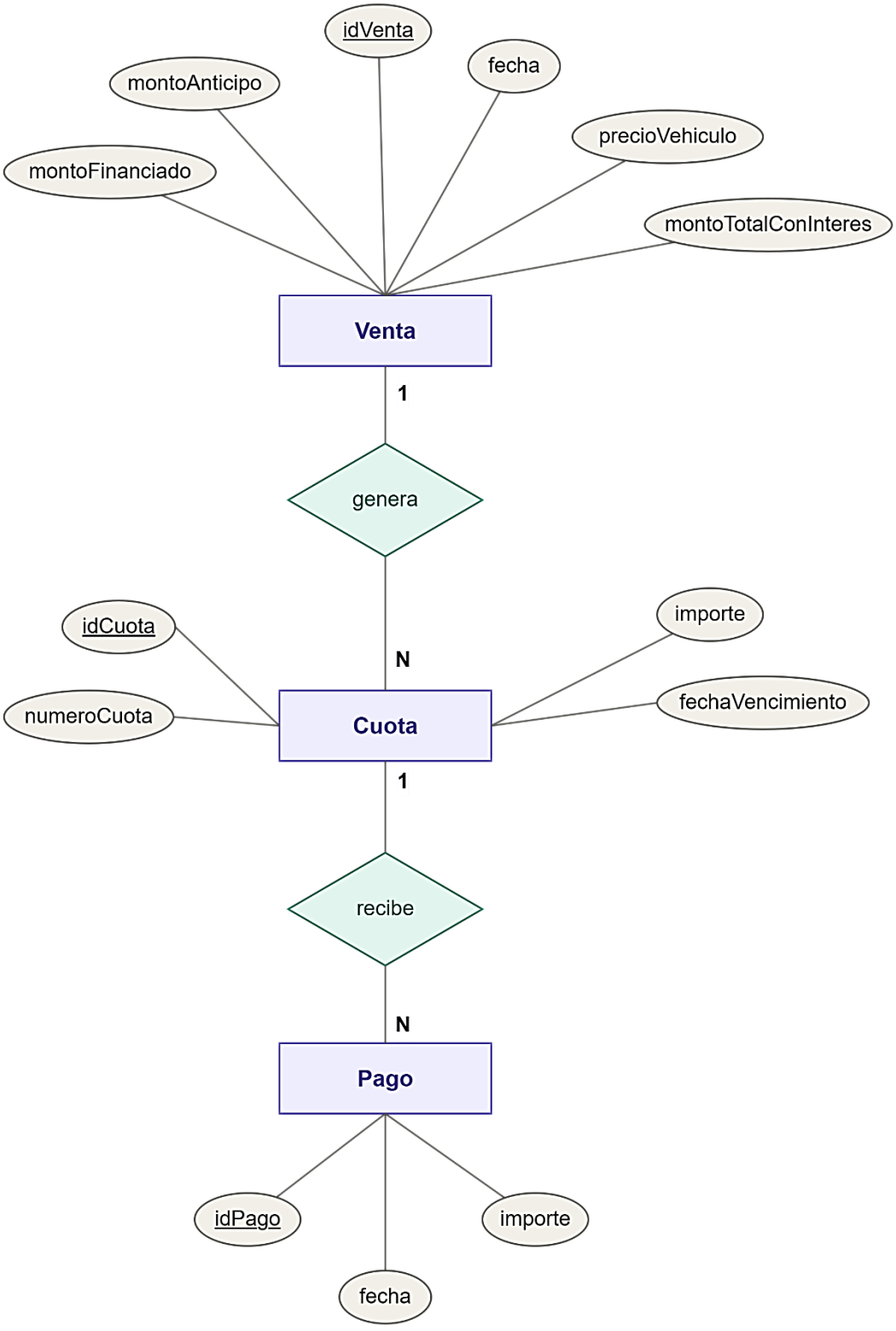
"Sistema de gestión de ventas y financiamiento para concesionaria"



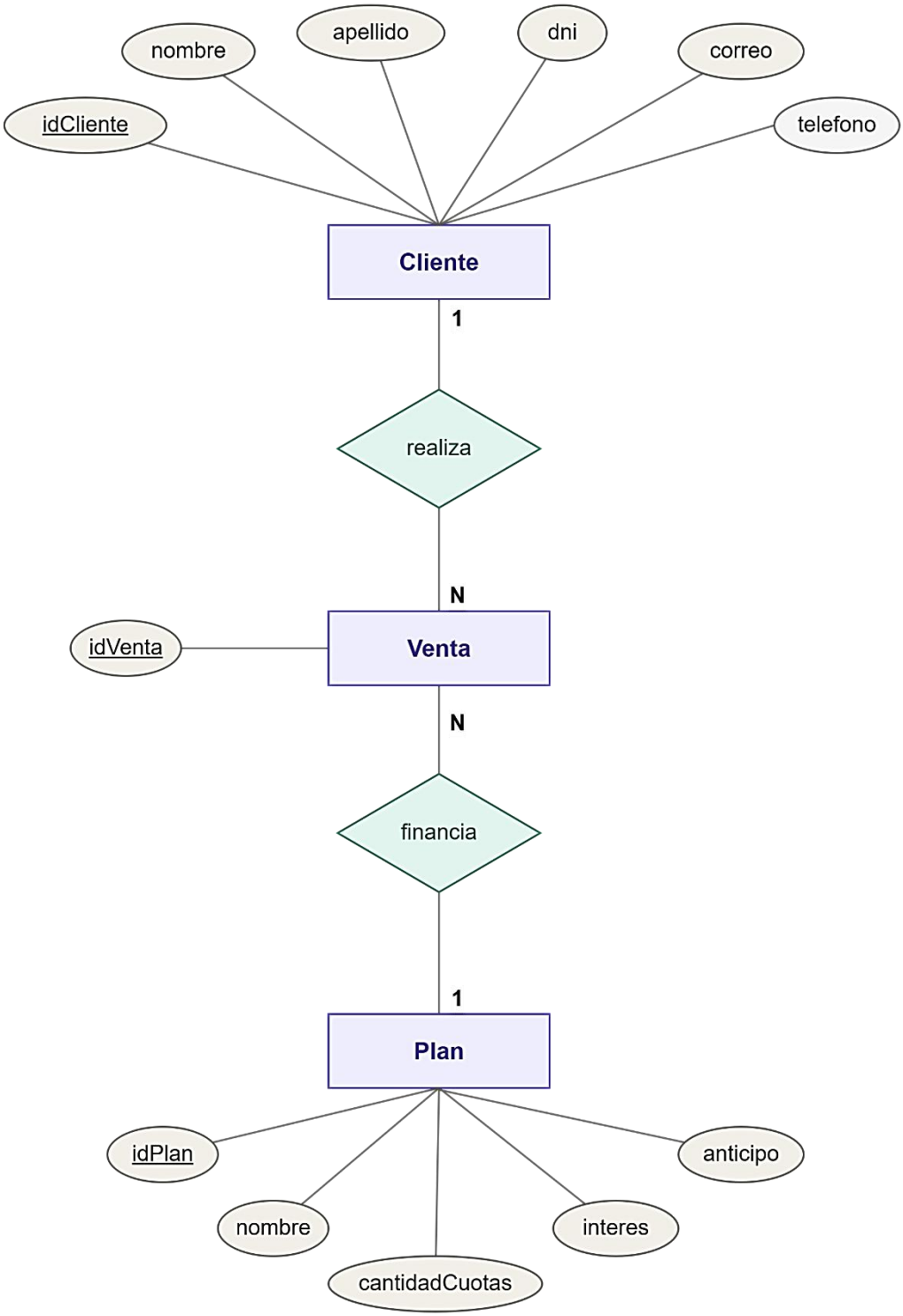
DER | VEHICULO - MARCA -TIPOVEHICULO - TIPOMOTOR



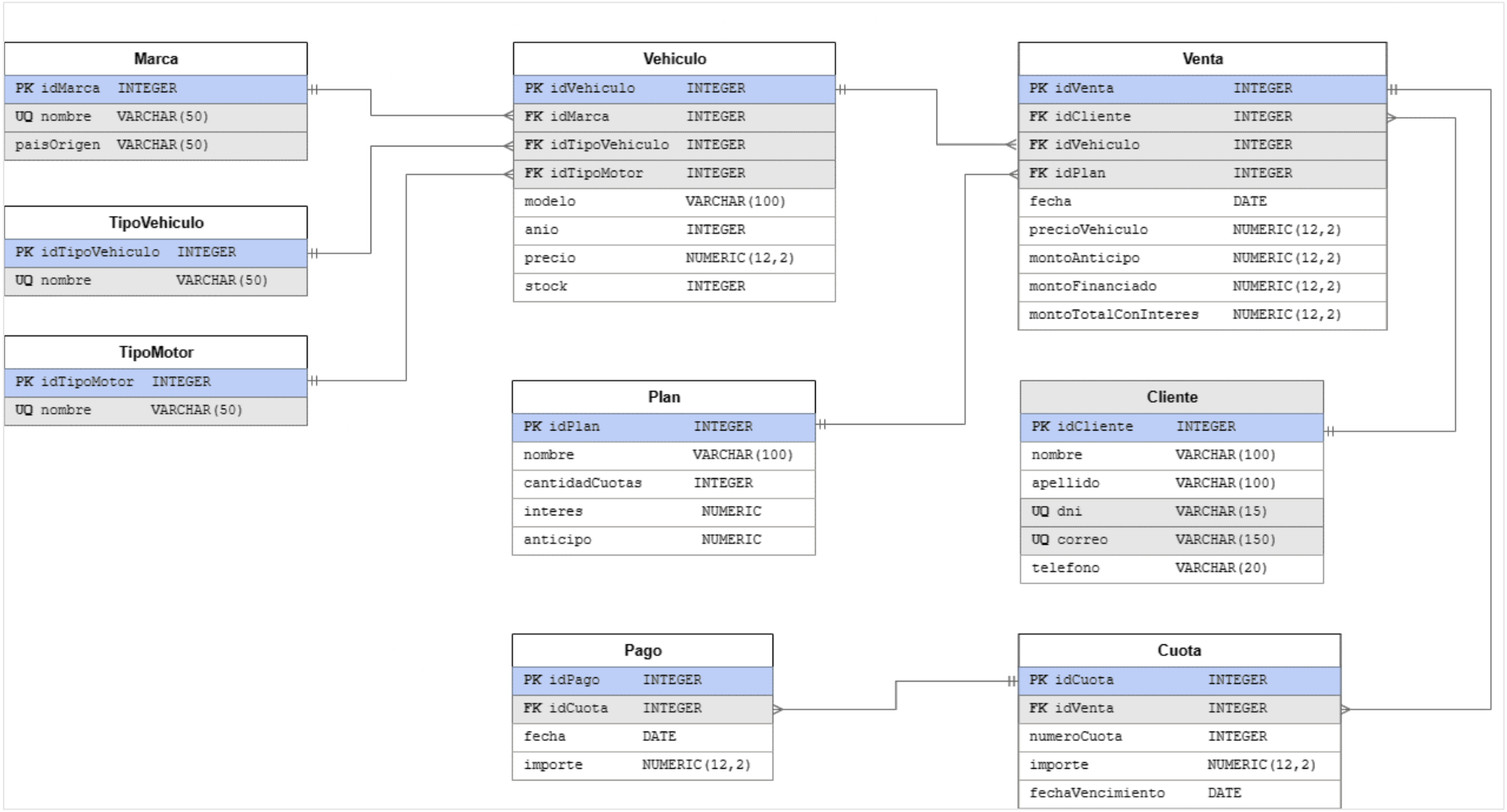
DER | VENTA - CUOTA - PAGO



DER | CLIENTE - VENTA - PLAN



2. MODELO RELACIONAL



3. *Clasificación*

- Las *Maestras*, que son las que guardan los datos de referencia con los cuales se realizarán las operaciones o cálculos necesarios. Son las primeras tablas en cargarse, en este caso, son las tablas: Marca, TipoVehiculo, TipoMotor, Cliente, Vehiculo y Plan.
- Las *Transaccionales*, que registran hechos que ocurren durante la operación y dependen de las maestras. Por ejemplo: Venta, Cuota y Pago.

PROCESAMIENTO DE TRANSACCIONES

Consiste en organizar como se registra cada operación o procedimiento y como se garantiza la integridad de los datos referenciales, los datos resultantes, o el estado de las operaciones mismas.

1. *Transacción*

Es el conjunto de operaciones que se aplican a los datos referenciales. Por ejemplo, al registrar una venta => se inserta una venta => se generan las cuotas => se descuenta el stock. Si falla la generación de las cuotas, la venta y el descuento de stock no pueden quedar guardados.

2. *Propiedades ACID*

Las propiedades ACID permiten asegurar la validez y calidad de los datos en los procesos de transacción, aun cuando ocurre una interrupción en la red o fallas en hardware. Estos atributos permiten leer y escribir al mismo tiempo sin que se corrompan o se pierdan los datos.

Propiedad	Qué garantiza	En el sistema
Atomicidad	Todo o nada.	Si falla una cuota, no queda la venta ni el stock descontado.
Consistencia	La transacción lleva la base de un estado válido a otro válido: se cumplen FK, UNIQUE y CHECK.	El stock nunca queda negativo.
Aislamiento	Las transacciones simultáneas no se ven entre sí a medio hacer.	Dos vendedores registran ventas al mismo tiempo.
Durabilidad	Lo confirmado con COMMIT se mantiene, aunque el servidor falle después.	Una venta confirmada no se pierde.

3. *Funciones PL/pgSQL y RAISE EXCEPTION*

PL/pgSQL es el lenguaje de PostgreSQL para escribir funciones con variables, condiciones (IF) y varias sentencias. Estructura básica:

```
CREATE OR REPLACE FUNCTION nombre(parámetros)
RETURNS tipo AS $$
DECLARE
  -- variables
BEGIN
  -- instrucciones
  RETURN resultado;
END;
$$ LANGUAGE plpgsql;
```

`RAISE EXCEPTION 'mensaje %', valor;` corta la función con un error; el % se reemplaza por el valor.

Una función se ejecuta dentro de la transacción de la consulta que la llama. Si lanzará un error, PostgreSQL revierte todo lo que la función hizo de forma automática e implícita (`ROLLBACK`); si el proceso finaliza de manera correcta se confirma confirma de forma automática (`COMMIT`).

4. Commit Y Rollback

El `COMMIT` es un comando que permite confirmar los cambios estableciendo una nueva versión. El `ROLLBACK` descarta los cambios, retornando a una versión anterior de la base de datos.

```
BEGIN;
  -- operaciones (INSERT, UPDATE...)
COMMIT;      -- o ROLLBACK; para deshacer todo
```

Si no se escribe `BEGIN`, cada sentencia suelta se ejecuta como una transacción propia y se confirma sola. En cambio, se busca que se ejecuten de forma secuencial como parte de una misma transacción. En funciones se implementan de forma implícita.

5. Concurrencia y bloqueo de registros

La concurrencia ocurre cuando hay varios usuarios operando al mismo tiempo, y puede suceder, por ejemplo, que realicen la misma operación. Por ejemplo: quedan dos ventas de un último vehículo (stock 1): las dos leen stock 1, las dos pasan la validación y las dos descuentan, y el stock termina en −1 lo que significa que se vendió una unidad que no existía.
Para evitar esto existen dos opciones:

- Bloqueo de fila: `SELECT ... FOR UPDATE` bloquea la fila del vehículo hasta que termina la transacción. La segunda venta espera, lee el stock ya actualizado (0) y es rechazada.
- Restricción `CHECK (stock >= 0)`: la segunda venta intenta dejar el stock en −1, la base lo rechaza y revierte la transacción.

GESTION DE PAGOS Y CONSULTAS DE DATOS DERIVADOS

1. Dato derivado

Son los datos que se pueden calcular a partir de otros almacenados en la base de datos. Por lo que no es necesario almacenarlos si se realizan al momento de la consulta. Guardarlo implica que los datos deben actualizarse cuando se requieran de manera correcta, si la actualización fuera incompleta podrían generarse inconsistencias.

Por ejemplo, en el sistema, el saldo de una cuota (importe − pagos recibidos) y su estado (pagada, pendiente o vencida) se calculan al momento de realizar la consulta. A su vez si el saldo, estuviera relacionado con la fecha actual, esta se actualizaría de forma automática.

Por ejemplo, en el sistema, el saldo de una cuota se calcula restando los pagos recibidos al importe total. El estado (pagada, pendiente o vencida) también puede calcularse a partir del saldo pendiente y la fecha de vencimiento al compararla con la fecha actual.

Así, el saldo se modifica cuando se registran pagos, mientras que el estado puede cambiar automáticamente con el paso del tiempo, sin necesidad de modificar los datos almacenados.

2. SUM, COUNT y GROUP BY

- `SUM()` suma los valores de una columna.
- `COUNT()` cuenta registros o valores, según lo que se indique.
- `GROUP BY` agrupa las filas que comparten un mismo valor y permite obtener un resultado por cada grupo.

Ejemplo: cantidad de ventas por marca.

```
SELECT m.nombre, COUNT(*) AS cantidad – - se identifica marca y cantidad
FROM venta v
JOIN vehiculo ve ON ve.idvehiculo = v.idvehiculo -- se relacionan venta y vehículo
JOIN marca m ON m.idmarca = ve.idmarca -- se relacionan venta y marca
GROUP BY m.nombre; -- una fila por marca, con la cantidad de ventas por vehículo
```

Toda columna del SELECT que no esté dentro de una función de agregación (SUM, COUNT...) tiene que figurar en el GROUP BY.

3. JOIN y LEFT JOIN

- `JOIN (o INNER JOIN)` devuelve únicamente las filas que tienen correspondencia en ambas tablas.
- `LEFT JOIN` devuelve todas las filas de la tabla izquierda y las coincidencias de la derecha. Si no hay coincidencia, las columnas de la derecha contienen NULL.

Ejemplo: mostrar todas las cuotas, incluso las que todavía no tienen pagos.

```
FROM cuota c
LEFT JOIN pago p ON p.idcuota = c.idcuota
```

Con `JOIN`, las cuotas sin pagos no aparecen. Con `LEFT JOIN`, sí aparecen, con `NULL` en las columnas de pago.

4. COALESCE

Devuelve el primer valor que no es `NULL`. `SUM` sobre una cuota sin pagos da `NULL`, no 0, y entonces `importe - NULL` también da `NULL`: el saldo desaparece.

`COALESCE (SUM (importe), 0)` lo convierte en 0.

5. CASE

Es un condicional dentro del `SELECT`: evalúa las condiciones en orden y devuelve el resultado de la primera que se cumple.

Ejemplo: determinar el estado de una cuota.

```
CASE
  WHEN saldo <= 0 THEN 'Pagada'
  WHEN fechavencimiento < CURRENT_DATE THEN 'Vencida'
  ELSE 'Pendiente'
END AS estado

/*

     WHEN: establece una condición.
     THEN: indica qué resultado devolver si se cumple.
     ELSE: define el resultado si ninguna condición anterior se cumple.
     END: finaliza el condicional.
*/
```

El orden importa porque primero se verifica si el saldo es menor o igual a cero y, si se cumple, la cuota figura como pagada, aunque su fecha de vencimiento ya haya pasado. Si se evaluara primero la fecha, la cuota figuraría como vencida sin verificar si ya está pagada.

6. Vistas

Una vista es una consulta guardada con un nombre que se utiliza como si fuera una tabla. No almacena los resultados de la consulta, sino que los obtiene al consultarla.

La vista `cuota_estado` calcula el saldo y el estado de todas las cuotas, y permite reutilizar esa información en distintos listados y gráficos.

```
SELECT *
FROM cuota_estado
WHERE estado = 'Vencida';

SELECT estado, COUNT(*)
FROM cuota_estado
GROUP BY estado; -- Agrupa las cuotas según su estado para contar cuántas hay en cada uno.
```

7. Saldos pendientes por cliente

Esta consulta calcula la deuda total de cada cliente sumando los saldos de sus cuotas y muestra únicamente aquellos cuya deuda es mayor que cero.

`GROUP BY` agrupa las cuotas que pertenecen al mismo cliente, `SUM()` suma los saldos de esas cuotas y `HAVING` muestra solo los clientes cuya suma es mayor que cero. A diferencia de `HAVING`, `WHERE` filtra las filas individuales antes de realizar la agrupación y la suma.

```
SELECT c.apellido, SUM(ce.saldo) AS deuda
FROM cuota_estado ce
JOIN venta v ON v.idventa = ce.idventa
JOIN cliente c ON c.idcliente = v.idcliente
GROUP BY c.idcliente, c.apellido
HAVING SUM(ce.saldo) > 0;
```

TABLAS

1. Entidades maestras

a) marca

```
-- Table: public.marca

-- DROP TABLE IF EXISTS public.marca;

CREATE TABLE IF NOT EXISTS public.marca
(
    idmarca integer NOT NULL DEFAULT nextval('marca_idmarca_seq'::regclass),
    nombremarca character varying(50) COLLATE pg_catalog."default" NOT NULL,
    paisorigen character varying(100) COLLATE pg_catalog."default" NOT NULL,
    CONSTRAINT marca_pkey PRIMARY KEY (idmarca),
    CONSTRAINT nombremarca UNIQUE (nombremarca)
        INCLUDE(nombremarca)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.marca
    OWNER to postgres;
```

b) vehículo

```
-- Table: public.vehiculo

-- DROP TABLE IF EXISTS public.vehiculo;

CREATE TABLE IF NOT EXISTS public.vehiculo
(
    idvehiculo integer NOT NULL DEFAULT nextval('vehiculo_idvehiculo_seq'::regclass),
    modelo character varying(100) COLLATE pg_catalog."default" NOT NULL,
    anio integer NOT NULL,
    precio numeric(12,2) NOT NULL,
```

```
    stock integer NOT NULL DEFAULT 0,
    idmarca integer NOT NULL,
    idtipovehiculo integer NOT NULL,
    idtipomotor integer NOT NULL,
    CONSTRAINT vehiculo_pkey PRIMARY KEY (idvehiculo),
    CONSTRAINT idmarca FOREIGN KEY (idmarca)
        REFERENCES public.marca (idmarca) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT idtipomotor FOREIGN KEY (idtipomotor)
        REFERENCES public.tipomotor (idtipomotor) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT idtipovehiculo FOREIGN KEY (idtipovehiculo)
        REFERENCES public.tipovehiculo (idtipovehiculo) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT condicionprecio CHECK (precio > 0::numeric),
    CONSTRAINT condicionstock CHECK (stock >= 0) NOT VALID
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.vehiculo
    OWNER to postgres;
```

c) cliente

```
-- Table: public.cliente

-- DROP TABLE IF EXISTS public.cliente;

CREATE TABLE IF NOT EXISTS public.cliente
(
    idcliente integer NOT NULL GENERATED BY DEFAULT AS IDENTITY ( INCREMENT 1 START 1
MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
    nombre character varying(100) COLLATE pg_catalog."default" NOT NULL,
    apellido character varying(100) COLLATE pg_catalog."default" NOT NULL,
    telefono character varying(20) COLLATE pg_catalog."default" NOT NULL,
    dni character varying(15) COLLATE pg_catalog."default" NOT NULL,
    correo character varying(150) COLLATE pg_catalog."default" NOT NULL,
    CONSTRAINT cliente_pkey PRIMARY KEY (idcliente),
    CONSTRAINT correo UNIQUE (correo)
        INCLUDE(correo),
    CONSTRAINT dni UNIQUE (dni)
        INCLUDE(dni)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.cliente
    OWNER to postgres;
```

d) tipoVehiculo

```
-- Table: public.tipovehiculo
```

```
-- DROP TABLE IF EXISTS public.tipovehiculo;

CREATE TABLE IF NOT EXISTS public.tipovehiculo
(
    idtipovehiculo integer NOT NULL DEFAULT
nextval('tipovehiculo_idtipovehiculo_seq'::regclass),
    nombretipo character varying(50) COLLATE pg_catalog."default" NOT NULL,
    CONSTRAINT tipovehiculo_pkey PRIMARY KEY (idtipovehiculo),
    CONSTRAINT nombretipo UNIQUE (nombretipo)
        INCLUDE(nombretipo)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.tipovehiculo
    OWNER to postgres;
```

e) tipoMotor

```
-- Table: public.tipomotor

-- DROP TABLE IF EXISTS public.tipomotor;

CREATE TABLE IF NOT EXISTS public.tipomotor
(
    idtipomotor integer NOT NULL DEFAULT
nextval('tipomotor_idtipomotor_seq'::regclass),
    nombremotor character varying(50) COLLATE pg_catalog."default" NOT NULL,
    CONSTRAINT tipomotor_pkey PRIMARY KEY (idtipomotor),
    CONSTRAINT nombremotor UNIQUE (nombremotor)
        INCLUDE(nombremotor)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.tipomotor
    OWNER to postgres;
```

f) plan

```
-- Table: public.plan

-- DROP TABLE IF EXISTS public.plan;

CREATE TABLE IF NOT EXISTS public.plan
(
    idplan integer NOT NULL DEFAULT nextval('plan_idplan_seq'::regclass),
    nombre character varying(100) COLLATE pg_catalog."default" NOT NULL,
    cantidadcuotas integer NOT NULL,
    porcentajeinteres numeric(5,2) NOT NULL,
    porcentajeanticipo numeric(5,2) NOT NULL,
    CONSTRAINT plan_pkey PRIMARY KEY (idplan)
)

TABLESPACE pg_default;
```

```
ALTER TABLE IF EXISTS public.plan
    OWNER to postgres;
```

2. *Procesamiento de transacciones*

g) ventas

```
-- Table: public.venta

-- DROP TABLE IF EXISTS public.venta;

CREATE TABLE IF NOT EXISTS public.venta
(
    idventa integer NOT NULL DEFAULT nextval('venta_idventa_seq'::regclass),
    idvehiculo integer NOT NULL,
    idplan integer NOT NULL,
    fecha date,
    preciovehiculo numeric(12,2),
    montoanticipo numeric(12,2),
    montofinanciado numeric(12,2),
    montototalconinteres numeric(12,2),
    idcliente integer NOT NULL,
    CONSTRAINT venta_pkey PRIMARY KEY (idventa),
    CONSTRAINT idcliente FOREIGN KEY (idcliente)
        REFERENCES public.cliente (idcliente) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
        NOT VALID,
    CONSTRAINT idplan FOREIGN KEY (idplan)
        REFERENCES public.plan (idplan) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT idvehiculo FOREIGN KEY (idvehiculo)
        REFERENCES public.vehiculo (idvehiculo) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.venta
    OWNER to postgres;

-- Trigger: trg_venta_inmutable

-- DROP TRIGGER IF EXISTS trg_venta_inmutable ON public.venta;

CREATE OR REPLACE TRIGGER trg_venta_inmutable
    BEFORE DELETE OR UPDATE
    ON public.venta
    FOR EACH ROW
    EXECUTE FUNCTION public.impedir_cambios_venta();
```

h) cuotas

```
-- Table: public.cuota

-- DROP TABLE IF EXISTS public.cuota;

CREATE TABLE IF NOT EXISTS public.cuota
(
    idcuota integer NOT NULL DEFAULT nextval('cuota_idcuota_seq'::regclass),
    idventa integer,
    numerocuota integer,
    importe numeric(12,2),
    fechavencimiento date,
    CONSTRAINT cuota_pkey PRIMARY KEY (idcuota),
    CONSTRAINT idventa FOREIGN KEY (idventa)
        REFERENCES public.venta (idventa) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT condicionimporte CHECK (importe > 0::numeric)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.cuota
    OWNER to postgres;
```

i) pagos

```
-- Table: public.pago

-- DROP TABLE IF EXISTS public.pago;

CREATE TABLE IF NOT EXISTS public.pago
(
    idpago integer NOT NULL DEFAULT nextval('pago_idpago_seq'::regclass),
    fecha date,
    importe numeric(12,2),
    idcuota integer,
    CONSTRAINT pago_pkey PRIMARY KEY (idpago),
    CONSTRAINT idcuota FOREIGN KEY (idcuota)
        REFERENCES public.cuota (idcuota) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
        NOT VALID
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.pago
    OWNER to postgres;
```

3. Lógica de negocios

j) ventas

```
-----------------------------------------------------
--                  funcion 1
--  validar_stock: verdadero si el vehículo existe y su stock es mayor a cero
-----------------------------------------------------

create or replace function validar_stock(idvehiculo INTEGER)
returns boolean as $$
    select exists (
        select 1 from vehiculo
        where vehiculo.idvehiculo = $1
          and stock > 0
    );
$$ language sql;


-----------------------------------------------------
--                  funcion 2
--  finalizar_venta: genera las cuotas y descuenta una unidad del stock
-----------------------------------------------------

create or replace function finalizar_venta(
    idventa INTEGER,
    idplan INTEGER,
    idvehiculo INTEGER
)
returns void as $$
begin
    -- generar las cuotas de la venta
    perform generar_cuotas(
        finalizar_venta.idventa,
        finalizar_venta.idplan
    );

    -- descontar una unidad del stock
    update vehiculo
    set stock = stock - 1
    where vehiculo.idvehiculo = finalizar_venta.idvehiculo;
end;
$$ language plpgsql;


-----------------------------------------------------
--                  funcion 3
--  registrar_venta: valida, inserta la venta con sus importes y la finaliza
-----------------------------------------------------

create or replace function registrar_venta(
    idcliente INTEGER,
    idvehiculo INTEGER,
    idplan INTEGER
)
returns venta as $$
declare
    nueva venta;
```

```
begin
    -- validar que el vehículo exista y tenga stock
    if not validar_stock(registrar_venta.idvehiculo) then
        raise exception 'El vehículo no existe o no tiene stock';
    end if;

    -- insertar la venta con los importes calculados
    insert into venta (
        idcliente, idvehiculo, idplan, fecha,
        preciovehiculo, montoanticipo, montofinanciado,
        montototalconinteres
    )
    select registrar_venta.idcliente, vehiculo.idvehiculo, plan.idplan,
           current_date, vehiculo.precio,
           round(vehiculo.precio * plan.porcentajeanticipo / 100, 2),
           round(vehiculo.precio * (1 - plan.porcentajeanticipo / 100), 2),
           round(vehiculo.precio * (1 - plan.porcentajeanticipo / 100)
                 * (1 + plan.porcentajeinteres / 100), 2)
    from vehiculo cross join plan
    where vehiculo.idvehiculo = registrar_venta.idvehiculo
      and plan.idplan = registrar_venta.idplan
    returning * into nueva;

    -- validar que el plan existía (si no, no se insertó ninguna fila)
    if nueva.idventa is null then
        raise exception 'El plan no existe';
    end if;

    -- generar las cuotas y descontar el stock
    perform finalizar_venta(nueva.idventa, registrar_venta.idplan,
                            registrar_venta.idvehiculo);

    return nueva;
end;
$$ language plpgsql;


-----------------------------------------------------
--  trigger: venta inmutable  (ejecutar UNA SOLA VEZ: si ya existe, da error)
--  Con el trigger activo no se pueden borrar ventas de prueba.
-----------------------------------------------------

create or replace function impedir_cambios_venta() returns trigger as $$
begin
    raise exception 'Una venta registrada no se puede modificar ni eliminar';
end;
$$ language plpgsql;

create trigger trg_venta_inmutable
before update or delete on venta
for each row execute function impedir_cambios_venta();
```

k) cuotas

```
create or replace function generar_cuotas(
```

```
    idventa INTEGER,
    idplan INTEGER
)
returns void as $$
begin
    insert into cuota (
        idventa,
        numerocuota,
        importe,
        fechavencimiento
    )
    select
        venta.idventa,
        numero,
        round(venta.montototalconinteres / plan.cantidadcuotas, 2),  -- redondea a dos
decimales
        (venta.fecha + numero * interval '1 month')::date            -- convierte el
resultado en fecha
    from venta
        join plan on plan.idplan = generar_cuotas.idplan
        cross join generate_series(1, plan.cantidadcuotas) as serie(numero)  -- una
fila por cuota
    where venta.idventa = generar_cuotas.idventa;
end;
$$ language plpgsql;


-------------------------------------------------------------------------
--                  vista 1
--  cuota_estado: saldo y estado de cada cuota (se calculan al consultar)
-------------------------------------------------------------------------

create or replace view cuota_estado as
select cuota.idcuota,
       cuota.idventa,
       cuota.numerocuota,
       cuota.importe,
       cuota.fechavencimiento,
       -- suma de los pagos de la cuota (0 si todavía no tiene ninguno)
       coalesce(sum(pago.importe), 0) as pagado,
       -- saldo pendiente = importe de la cuota - pagos recibidos
       cuota.importe - coalesce(sum(pago.importe), 0) as saldo,
       -- estado: sin saldo = pagada; con saldo y fecha pasada = vencida; el resto =
pendiente
       case
           when cuota.importe - coalesce(sum(pago.importe), 0) <= 0 then 'Pagada'
           when cuota.fechavencimiento < current_date then 'Vencida'
           else 'Pendiente'
       end as estado
from cuota
left join pago on pago.idcuota = cuota.idcuota   -- left join: incluye las cuotas sin
pagos
group by cuota.idcuota, cuota.idventa, cuota.numerocuota, cuota.importe,
cuota.fechavencimiento;
```

l) pagos

```
- ----------------------------------------------------
--                  funcion 1
--  saldo_cuota: devuelve lo que falta pagar de una cuota
--               (importe de la cuota - pagos recibidos)
------------------------------------------------------

create or replace function saldo_cuota(cuota_elegida INTEGER)
returns numeric as $$
declare
    importe_cuota numeric;   -- importe total de la cuota
    total_pagado  numeric;   -- suma de lo ya pagado de esa cuota
begin
    -- traer el importe de la cuota; "for update" la bloquea hasta que termine
    -- la transacción, así dos pagos simultáneos no se pisan entre sí
    select importe into importe_cuota
    from cuota
    where idcuota = cuota_elegida
    for update;

    -- si no se encontró la cuota, cortar con error
    if not found then
        raise exception 'La cuota % no existe', cuota_elegida;
    end if;

    -- sumar los pagos anteriores (coalesce devuelve 0 si todavía no hay ninguno)
    select coalesce(sum(importe), 0) into total_pagado
    from pago
    where idcuota = cuota_elegida;

    -- saldo = importe de la cuota - pagos recibidos
    return importe_cuota - total_pagado;
end;
$$ language plpgsql;


------------------------------------------------------
--                  funcion 2
--  registrar_pago: valida el importe, controla que no supere el saldo,
--                  inserta el pago con la fecha de hoy y lo devuelve
------------------------------------------------------

create or replace function registrar_pago(cuota_elegida INTEGER, importe_a_pagar
numeric)
returns pago as $$
declare
    saldo_pendiente numeric;   -- lo que falta pagar de la cuota
    pago_nuevo      pago;      -- el pago recién creado
begin
    -- 1) el importe tiene que ser mayor a cero
    if importe_a_pagar <= 0 then
        raise exception 'El importe debe ser mayor a cero';
    end if;

    -- 2) calcular el saldo (también valida que la cuota exista)
    saldo_pendiente := saldo_cuota(cuota_elegida);
```

```
    -- 3) el pago no puede superar el saldo pendiente
    if importe_a_pagar > saldo_pendiente then
        raise exception 'El pago (%) supera el saldo pendiente (%)',
                        importe_a_pagar, saldo_pendiente;
    end if;

    -- 4) insertar el pago con la fecha de hoy y devolverlo
    insert into pago (idcuota, fecha, importe)
    values (cuota_elegida, current_date, importe_a_pagar)
    returning * into pago_nuevo;

    return pago_nuevo;
end;
$$ language plpgsql;
```